

Inigo Quilez :: articles :: SDF Bounding Volumes - 2019



Intro

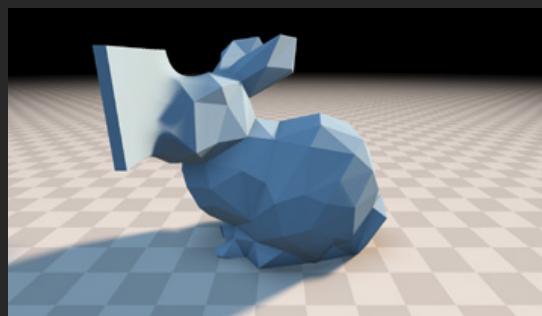
When **raymarching SDFs**, or when combining regular GL/DX/VK rasterization with SDF for shadows, occlusion or collision, often times one finds that the SDF evaluation is too expensive. That can be even more true for SDFs that are fully made of **procedural primitives** or when they are evaluated for a 3D mesh with lots of triangles.

In my Shadertoy experiments, often times I embrace the limitation and design my scenes and world accordingly, which means I often create simple shapes rather than the complex ones I'd love to do. However, it is not a bad idea to use simple Bounding Volumes around complex SDF objects in order to avoid evaluating the full SDF when it's not needed. This reduces complexity of the overall SDF evaluation and works pretty well in most platforms (specially those in which branching is not too expensive).

This article describes a couple simple techniques I have used in the past that work well, and some quick experiments on more advanced ones involving acceleration structures that can inspire you to build more sophisticated ones perhaps.



Simple Bounding Volume for raymarching



Bounding Volume Hierarchy for raymarching



KD-Tree for raymarching

Basic Bounding Volumes

So, lets take a complex object (say, a character), which could be defined as follows:

```
float sdCharacter( in vec3 pos, in float minDist )
{
    float d1 = sdHead( pos ); // expensive SDF
    float d2 = sdBody( pos ); // expensive SDF
    float d3 = sdLegs( pos ); // expensive SDF
    float d4 = sdArms( pos ); // expensive SDF

    float dT = min(min(d1,d2),min(d3,d4));

    if( dT < minDist ) minDist = dT;

    return minDist;
}
```

First, note I have used a branch in the last distance comparison to exemplify that actually often times you want to track not only proximity but material id and surfacing properties, which needs a branch and cannot be done with the simple union operator `min()`.

Now we are going to doing the simplest thing we can imagine: define a sphere that completely bounds the character, and check the distance to that sphere first before we evaluate any of the character parts. If the current closest distance to the global SDF before evaluating the character, `minDist`, is already smaller than the distance to the bounding sphere, there's no need to evaluate any of the character's SDF parts at all, since none of the partial distances `d1` to `d4` can possibly be smaller than `minDist`. Let's see what it'd look like in code:

```
float sdCharacter( in vec3 pos, in float minDist )
{
    // early skip
    float dB = sdSphere( pos, boundingSphereRadius );
```

```
if( dB > minDist ) return minDist;

// regular evaluation
float d1 = sdHead( pos ); // expensive SDF
float d2 = sdBody( pos ); // expensive SDF
float d3 = sdLegs( pos ); // expensive SDF
float d4 = sdArms( pos ); // expensive SDF

float dT = min(min(d1,d2),min(d3,d4));

if( dT < minDist ) minDist = dT;

return minDist;
}
```

A trick I want to share for those hacking Shadertoy SDFs, but not worth doing in other contexts, is the following: probably some primitive of some of the character parts can be repurposed as the actual bounding volume, after expansion. Say `sdBody()` had a big sphere evaluation inside of it that defines the shape of the belly, or torso, or something. Then, we could evaluate that `sdSphere` right-away as first thing in `sdCharacter()`, and then add a fixed distance that would inflate the sphere's radius to match our bounding needs. The addition would be done only for the bounding volume evaluation, the actual `sdSphere()` call inside `sdBody()` would stay unchanged since otherwise we'd naturally be changing the shape of the character. This of course works best when the SDF we are bounding is not very modularized into different sub-SDFs as this one example above is, since that optimization would break quite a good deal of code modularity. But sometimes it can be helpful, and I have actually used this technique to get a 8x speed up in the rendering of the expensive character in the video below (which you can also find live and with full source code in Shadertoy: <https://www.shadertoy.com/view/3lsSzf>).



Of course, the bound volumes don't need to be closed shapes like spheres and boxes, they can also be half spaces. That can be especially handy if the SDF to be bound is infinite, such as ground geometries. In that case, a horizontal plane right above the highest feature of the ground (pebble, rock, bush) can serve as a very effective bounding volume to prevent evaluation of the complex ground SDF when the evaluation is happening a few meters above it and we've already found some close geometry such as a tree or a character.

Recursive Bounding Volumes

Naturally, like many things in computer graphics, a problem can be split into smaller problems, allowing it to scale to bigger datasets. In this case, nothing stops us from going beyond adding a bounding volume to `sdCharacter()` and adding bounding volumes as well to `sdHead()`, `sdBody()` and all other sub-parts.

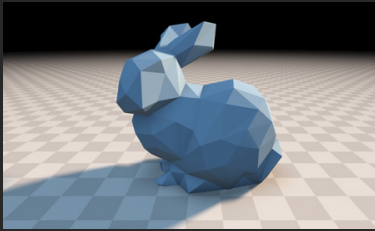
Of course, evaluating bounding volumes does add some performance cost in the cases where the SDF it bounds ends up needing to be evaluated. So one has to balance the cost of doing the bounding volume evaluation relative to the raw cost of the SDF it is trying to optimize.

Also, even if a considerable amount of computation was being saved on average, the cost of taking branches (if statements) and the extra fetching and storing the data needed to describe the bounding volume can be very expensive on some platforms (for example, on the web with WebGL or on mobile). So there's a platform dependent limit to how deep in recursion one should go with the bounding volumes and how granular the bounding strategy needs to be.

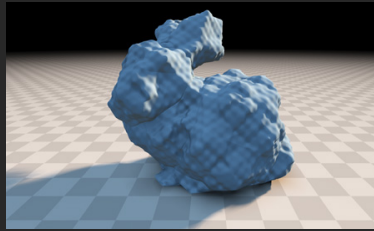
Bounding Volumes Hierarchies

One extreme case of recursive bounding volumes is when the complexity of the SDF is so high that no matter what, a really deep recursive bounding volume system is needed. There are multiple and well understood Bounding Volume Hierarchy algorithms that one can pick from, both using a `stack()` to implement automatic recursion, without a stack by using a restart-from-root approach, and without stack and by using skip pointers or parent pointers. These are probably outside the scope of this article, but they are not difficult to implement, and can make some otherwise complex SDFs run at full framerate and full screen, such as this (tiny) triangle mesh. This is an example of a simply

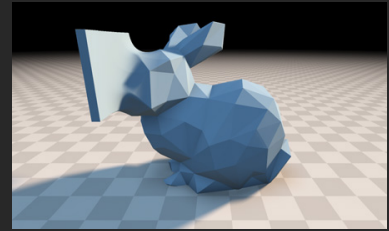
BVH applied to the Stanford Bunny mesh used for raw raymarching, noise free **single-ray soft shadows**, inexpensive occlusion (Shadertoy style rendering, basically), **displacement** and **soft blending operators**. It runs full framerate in my GeForce 1070 based laptop, using a single fragment shader to do all rendering (again, Shadertoy style). The mesh and BVH data is stored in a Shader Storage Buffer, so I couldn't share it as an actual Shadertoy example.



Soft shadows produced by an exact SDF



Displacement



Smooth minimum

Besides the nice shading and lighting tricks, one advantage of an SDF representation for a mesh is that one can do easy edge antialiasing and more importantly, it opens the doors for research in geometric filtering for analytic LOD, since the SDF of a mesh provides a volumetric representation.

The code wasn't particularly optimized, I wrote the whole demo in a night, but it runs well enough as a simple fragment shader as I said (no compute):

```
float sdMesh( in vec3 pos, in float minDist )
{
    stack_reset();
    int currentNode = 0;
    for( ;; )
    {
        Node n = data.node[ currentNode ];

        // if we hit bounding volume, go down the tree
        if( sdBox( pos, n.bbox ) < minDist )
        {
            // intersect triangles in this node
            if( n.numTriangles > 0 )
            {
                minDist = sdTriangles( pos, n.trianglesOffset, n.numTriangles, minDist );
            }

            // traverse children, closest one first
            int closest = (pos[n.splitAxis] < center(n.bbox)[n.splitAxis]) ? 0 : 1;
            stack_push( Data(n.childOffset+1-closest) );
            stack_push( Data(n.childOffset+ closest) );
        }

        // get next node, if any
        if( stack_is_empty() ) break;
        currentNode = stack_pop();
    }
}
```

```
    return minDist;  
}
```

Please note this code is just given as a basic reference to give you an idea of the kind of algorithm used, and it's by no means optimized. For example, in practice one can traverse big parts of the hierarchy without ever pushing and popping nodes into the stack by rearranging the traversal in a nested loop. Also, all the node data such as numTriangles and splitAxis are just bits in a shared integer field, and trianglesOffset and childOffset are of course a shared integer as well. And of course the bounding boxes can be quantized. Lastly, you can perform the whole mesh SDF evaluation with a single square root, since both the sdBox() and sdTriangle() functions allow for taking it as a common factor. Note however how in this implementation the tree is binary and the children are allocated in pairs in order to save on pointer/offsets.

In practice, much of the performance is determined by the tree construction algorithm and considerations such as the SAH (Surface Area Heuristic) that determines how to split a node into children, amount of children overlap that is allowed or encouraged, the maximum number of triangles allowed in a node, etc.

Spatial Subdivision

One problem with BVH is that each node needs to store its bounding volume, which can be very memory consuming even when quantized heavily. One way to improve that situation is to use Spatial Subdivision schemas instead, which don't partition the data-set but space. So, technically they don't belong in an article called "Bounding Volumes", but since they are relevant and I used a second night to experiment with them, I'll talk quickly about it now, if only as an introduction.

Grids, Octrees and KD-Trees are the best known Spatial Subdivision structures. From these, only Octrees and KD-Trees are hierarchical, which we need. KD-Trees are binary trees, so their traversal logic is simpler than that for Octrees. While in an Octree traversal one needs to decide the order priority of the 8 children, in a KD-tree there's only two and usually a simple comparison or sign bit extraction suffices to determine the best children to proceed with.

The advantage of Spatial Subdivision structures over Bounding Volume Hierarchies is that they offer a smaller memory footprint and bandwidth. The cons, however, are that they are more expensive to build, AND require a thicker stack in the shader code during runtime traversal. A thicker stack consumes more registers and can kill parallelism in the thread group, if implemented in the GPU. Techniques can be used to amortize the cost of traversal among tiles of pixels by using compute shaders, but at that point we are asking for problems when rays are shot randomly instead of in right packets (unless, again, an even more sophisticated layer of point evaluation sorting is added on top, again

outside the scope of this article).

The following is a simplified comparison between the trade-offs in the Node and Stack structures for both methods:

	Node Data	Stack Data
BVH	BBox, PointerOffset, Flags	PointerOffset, Flags
KD-Tree	NodeID	BBox, NodeID

Of course there are hybrid acceleration structures such as BIH, but that night I came up with an interesting way for using KD-Trees without having to pay the cost of the BBox (or split plane location) storage in the stack. The idea was to sacrifice tree quality and use a simple procedural space splitting criteria instead of sophisticated SAH during tree construction. This should allow, from the Node ID/address alone to reconstruct its bounding box completely in the shader. Meaning all that I'd need to push to the stack would be an integer, not a whole bbox. The cons and pros of this method are:

Cons:

- Lower quality tree
- More compute during traversal (need to build the bounding boxes procedurally)

Pros:

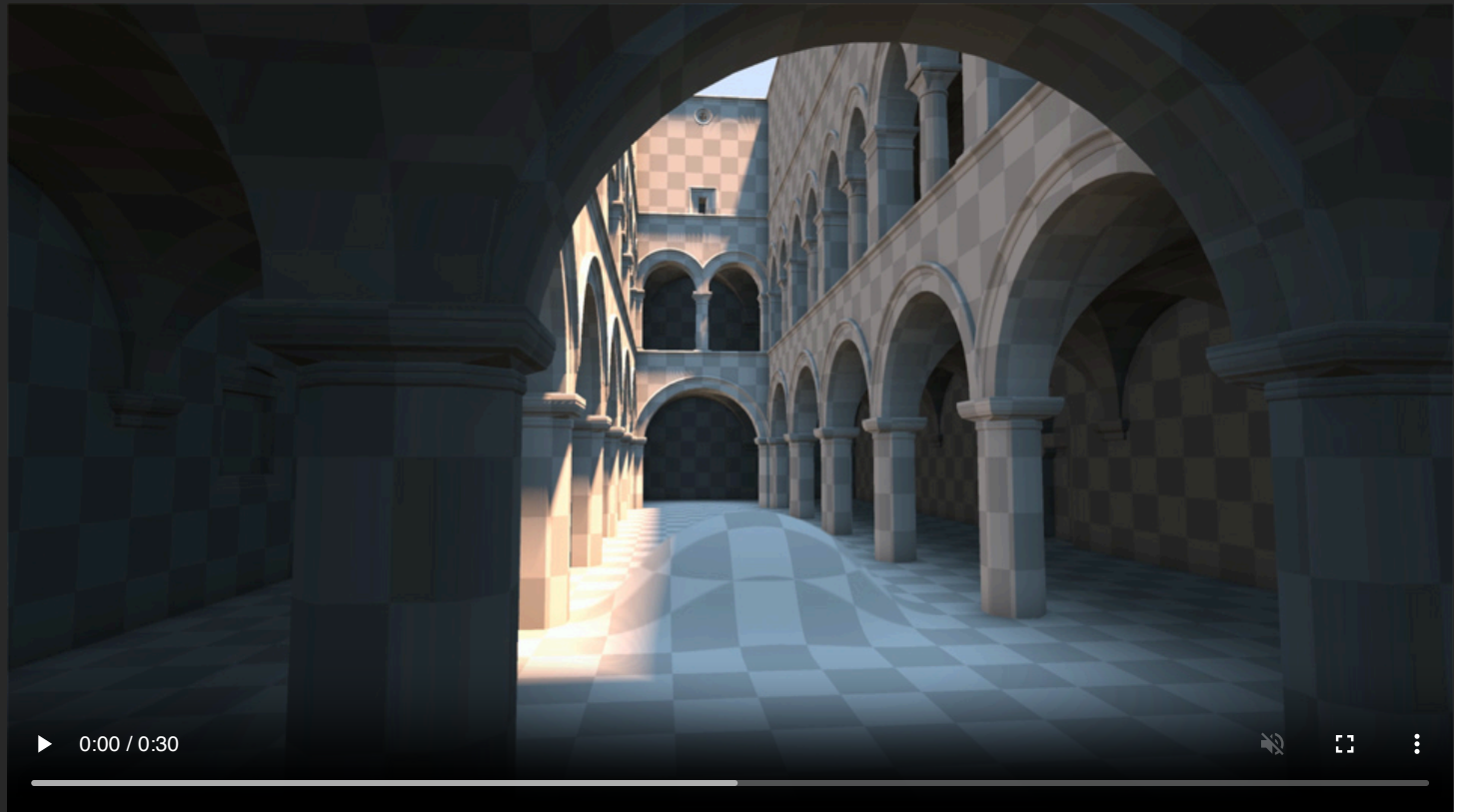
- Think stack like BVH, less register pressure
- Faster build times (no SAH!)

In my experiment I was able to build the bounding boxes from the address of each node by interpreting it a binary string where each bit indicates a left or right branch taken down the tree. I implemented this by using GLSL's `findMSB()` call and some branchless bit manipulations (as many as levels of depth were allowed in my tree).

I coded the procedural KD-tree in a second night of work and this time I implemented a more proper KD-tree traversal loop, which I had experience with from (now 15 years old!) past [real-time SIMD packet raytracing](#) work. Of course traversing and looking for intersections is different than traversing looking for closest distance, but the code is similar enough so these were not unknown waters this time.

For a single night of work, the technique was not a failure, but it wasn't also super successful, I think I'd need to spend more time in this problem space in order to get a good traversal implementation, and maybe do it as a compute shader rather than a fragment shader. But, it did work to some degree, at least I was able to raymarch primary rays and soft shadows for some models like the Sponza atrium (downloaded from Morgan McGuire's

Computer Graphics Archive <https://casual-effects.com/data>)



KD-Tree driven raymarched SDF, with soft shadows, object blending, occlusion and distance driven moss.

The performance was pretty bad though, on the single digit framerate in my laptop computer + GeForce 1070. Still, you can see nice soft shadows, some contact shadowing/occlusion and a sphere that I added to the scene to show this was an SDF, so for a quick experiment I think this was a good outcome.



Hit map (triange+box sdf evaluations per mesh SDF evaluation)

Overall, I think there's lots of potential for SDF evaluation using the same acceleration structures used for regular raycasting, and the benefit is that one can evaluate proximity to geometry easily, which has multiple purposes beyond shadowing tricks - for example, you can use it to drive vegetation growth and wetness or snow, just to name a few shading ideas, as shown in the image above where the proximity to other geometry drives the green surface coloring. I was told that the acceleration structures for HW raytracing in current GPUs wouldn't be efficient for SDF evaluation, but I think they don't need to be - you can get a lot of global proximity information with one or two queries per pixel and do lots of cool things with them that would require dozens of regular rays. So I feel we have a 10x performance hit we can take for SDF evaluation and still be competitive with raycast based methods?

inigo quilez - learning computer graphics since 1994